

Benchmark-Bericht: Retrieval-Performance & Infrastrukturkosten

Hinweis für das RAG-Parsing: Dieses Dokument dient als synthetischer Testdatensatz zur Validierung von Tabellen-Extraktionsalgorithmen (z. B. Unstructured, LlamaParse) und LLM-basierten Tabellenbeschreibungen innerhalb der Daten-Ingestion-Pipeline.

1. Evaluierung der Retrieval-Performance

Für die Optimierung unseres RAG-Systems (Retrieval-Augmented Generation) wurden verschiedene Kombinationen aus Embedding-Modellen, Chunk-Größen und Overlap-Verhältnissen getestet. Ziel war es, die Trefferquote (Hit Rate) sowie die Relevanz der abgerufenen Dokumente (Mean Reciprocal Rank, MRR) zu maximieren, während die Latenz in einem produktionsreifen Rahmen bleibt.

Die Tests wurden auf einem standardisierten Validierungsset von 1.500 domänenspezifischen Fragen durchgeführt. Die Ergebnisse verdeutlichen, dass größere Chunk-Größen zwar den Kontext verbessern, aber die Retrieval-Latenz signifikant erhöhen.

Modell-ID / Name	Chunk-Größe	Overlap (%)	Hit Rate @5	MRR @5	Latenz (avg)
text-embedding-3-small	256 Tokens	10%	0,78	0,68	42 ms
text-embedding-3-small	512 Tokens	15%	0,82	0,71	48 ms
text-embedding-3-large	512 Tokens	15%	0,89	0,79	74 ms
bge-m3-dense-only	512 Tokens	20%	0,86	0,75	115 ms
cohere-embed-multilingual-v3	512 Tokens	20%	0,92	0,84	61 ms
cohere-embed-multilingual-v3	1024 Tokens	10%	0,91	0,81	95 ms

Wie in der obigen Tabelle ersichtlich, liefert das Modell cohere-embed-multilingual-v3 bei einer Chunk-Größe von 512 Tokens und 20% Overlap das beste Verhältnis aus Genauigkeit (Hit Rate @5 = 0,92) und Latenz (61 ms). Die Erhöhung der Chunk-Größe auf 1024 Tokens führte zu einem leichten Performance-Abfall, was auf eine Verwässerung des Fokus im Vektorraum hindeutet.

2. Infrastruktur-Skalierung und Vektordatenbanken

Neben der reinen Retrieval-Qualität spielen die Betriebskosten und der Durchsatz (Queries per Second, QPS) eine entscheidende Rolle für den produktiven Rollout. Es wurden vier führende Vektordatenbank-Setups auf Basis einer geschätzten Ziel-Datenmenge von 10 Millionen Vektoren evaluiert.

Vektor-DB	Cluster-Konfiguration	RAM / Node	Max. Durchsatz	Kosten / Monat
Qdrant Cloud	2x AWS m6i.xlarge (Managed)	16 GB	450 QPS	480,00 €
Pinecone Enterprise	3x S1 Pods (Replica Setup)	24 GB	600 QPS	820,00 €
Milvus Distributed	4x QueryNodes + 2x ProxyNodes	32 GB	1.450 QPS	1.250,00 €
pgvector (RDS)	1x db.r6g.2xlarge (Postgres 16)	64 GB	310 QPS	650,00 €

Die Wahl der Vektordatenbank beeinflusst direkt die Systemarchitektur. Während Milvus den höchsten Durchsatz für parallele Nutzeranfragen bietet, stellt Qdrant Cloud die kosteneffizienteste Variante für das aktuelle Projektstadium dar. pgvector auf einer AWS RDS-Instanz ist eine starke Option, sofern eine strikte relationale Datenkonsistenz und eine vereinfachte Infrastruktur gefordert sind.

3. Fazit für die Ingestion-Pipeline

Für die finale Pipeline-Architektur wird empfohlen, Tabellen vor dem Chunking explizit zu isolieren und über ein separates LLM (z. B. GPT-4o-mini) textuell beschreiben zu lassen. Diese generierte Beschreibung (z. B. „Tabelle zeigt den Vergleich von Embedding-Modellen...“) sollte anschließend zusammen mit den Tabellen-Metadaten in den Vektorraum indexiert werden, um ein semantisches Auffinden komplexer Datenstrukturen zu garantieren.